GIT, GESTION DE PROJET ET AGILITÉ · JOUR 2 SUR 5 · MARDI 27 OCTOBRE

Mardi · Branches, conflits et GitHub

Hier, tous vos commits se suivaient en ligne droite. Ça va tant qu'on fait une chose à la fois. Mais si vous voulez tester une idée sans casser ce qui marche, ou si trois personnes travaillent en même temps, il faut des lignes de travail séparées : les **branches**.

L'après-midi, on met vos dépôts en ligne sur GitHub.

1. Créer une branche

Une branche, c'est une ligne de travail parallèle. Vous faites vos essais dessus, et `main` reste intacte. Si l'essai est concluant, on le fusionne dans `main`. Sinon, on jette la branche.

On repart d'un petit dépôt de recettes :

Un dépôt de départ

```
$ git log --oneline
b5bb587 (HEAD -> main) Ajoute la recette des crêpes
bed3ced Ajoute le README
$ git branch
* main
```

`git branch` liste les branches. L'étoile montre celle où vous êtes. On en crée une nouvelle et on s'y place, en une seule commande :

Travailler sur une branche

```
$ git switch -c gaufres
Switched to a new branch 'gaufres'
$ printf "Farine\nBeurre fondu\nSucre\n" > gaufres.txt
$ git add gaufres.txt
$ git commit -m "Ajoute la recette des gaufres"
[gaufres 8e27dd7] Ajoute la recette des gaufres
 1 file changed, 3 insertions(+)
 create mode 100644 gaufres.txt
$ git log --oneline
8e27dd7 (HEAD -> gaufres) Ajoute la recette des gaufres
b5bb587 (main) Ajoute la recette des crêpes
bed3ced Ajoute le README
```

Maintenant, revenez sur `main` et regardez le dossier :

Le fichier disparaît… sur main

```
$ git switch main
Switched to branch 'main'
$ ls
README.md
crepes.txt
$ git log --oneline --graph --all
* 8e27dd7 (gaufres) Ajoute la recette des gaufres
* b5bb587 (HEAD -> main) Ajoute la recette des crêpes
* bed3ced Ajoute le README
```

`gaufres.txt` a disparu ! Rassurez-vous : il existe toujours, mais seulement sur la branche `gaufres`. Quand vous changez de branche, Git remet le dossier dans l'état de cette branche. `git log --graph --all` dessine toutes les branches : on voit que `gaufres` a un commit d'avance.

Avant de changer de branche, commitez votre travail en cours. Sinon Git refuse le changement, ou emporte vos modifications avec vous.

2. Fusionner une branche

On se place sur la branche qui doit **recevoir** les changements (ici `main`), et on fusionne :

Fusion simple (fast-forward)

```
$ git merge gaufres
Updating b5bb587..8e27dd7
Fast-forward
 gaufres.txt | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 gaufres.txt
$ ls
README.md
crepes.txt
gaufres.txt
$ git branch -d gaufres
Deleted branch gaufres (was 8e27dd7).
```

Fast-forward : `main` n'avait pas bougé depuis la création de la branche, alors Git avance simplement `main` jusqu'au dernier commit de `gaufres`. Une fois fusionnée, la branche ne sert plus : on la supprime avec `git branch -d`.

Si les deux branches ont avancé chacune de leur côté, Git ne peut plus « avancer ». Il crée un **commit de fusion**, qui réunit les deux histoires :

Deux branches qui ont divergé

```
$ git switch -c dessert
Switched to a new branch 'dessert'
$ echo "Mousse au chocolat : 200 g de chocolat, 6 œufs" > mousse.txt
$ git add mousse.txt && git commit -m "Ajoute la mousse au chocolat"
[dessert 8470f22] Ajoute la mousse au chocolat
 1 file changed, 1 insertion(+)
 create mode 100644 mousse.txt
$ git switch main
Switched to branch 'main'
$ echo "Toutes les recettes sont pour 4 personnes." >> README.md
$ git commit -am "Précise le nombre de personnes"
[main 67c3f52] Précise le nombre de personnes
 1 file changed, 1 insertion(+)
$ git merge dessert
Merge made by the 'ort' strategy.
 mousse.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 mousse.txt
$ git log --oneline --graph
*   27189c4 (HEAD -> main) Merge branch 'dessert'
|\
| * 8470f22 (dessert) Ajoute la mousse au chocolat
* | 67c3f52 Précise le nombre de personnes
|/
* 8e27dd7 Ajoute la recette des gaufres
* b5bb587 Ajoute la recette des crêpes
* bed3ced Ajoute le README
```

Dans la vraie vie, Git ouvre ici l'éditeur pour le message du commit de fusion. Le message proposé (`Merge branch 'dessert'`) convient très bien : enregistrez et fermez (`Ctrl+O`, `Entrée`, `Ctrl+X` dans nano). Pour sauter cette étape : `git merge --no-edit dessert`.

3. Les conflits

Un conflit arrive quand deux branches modifient **la même ligne** du même fichier. Git ne peut pas deviner laquelle garder : il vous demande de choisir. Ce n'est pas une erreur grave, c'est une question.

Provoquer un conflit

```
$ git switch -c sans-lactose
Switched to a new branch 'sans-lactose'
$ sed -i "s/Lait/Lait d'avoine/" crepes.txt
$ git commit -am "Remplace le lait par du lait d'avoine"
[sans-lactose 0e46dc9] Remplace le lait par du lait d'avoine
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git switch main
Switched to branch 'main'
$ sed -i 's/Lait/Lait entier/' crepes.txt
$ git commit -am "Précise le lait entier"
[main 2cbdf8b] Précise le lait entier
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git merge sans-lactose
Auto-merging crepes.txt
CONFLICT (content): Merge conflict in crepes.txt
Automatic merge failed; fix conflicts and then commit the result.
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   crepes.txt

no changes added to commit (use "git add" and/or "git commit -a")
$ cat crepes.txt
Farine
Œufs
<<<<<<< HEAD
Lait entier
=======
Lait d'avoine
>>>>>>> sans-lactose
```

Git a écrit les deux versions dans le fichier, encadrées par des marqueurs :

- entre `<<<<<<< HEAD` et `=======` : votre version, celle de la branche où vous êtes (`main`) ;
- entre `=======` et `>>>>>>> sans-lactose` : la version de la branche que vous fusionnez.

Pour résoudre : ouvrez le fichier, gardez ce que vous voulez (une version, l'autre, ou un mélange), **supprimez les trois lignes de marqueurs**, enregistrez. Puis `git add` et `git commit`.

Résoudre le conflit

```
$ printf "Farine\nŒufs\nLait entier (ou lait d'avoine)\n" > crepes.txt
$ git add crepes.txt
$ git commit --no-edit
[main 63a3407] Merge branch 'sans-lactose'
$ git log --oneline --graph
*   63a3407 (HEAD -> main) Merge branch 'sans-lactose'
|\
| * 0e46dc9 (sans-lactose) Remplace le lait par du lait d'avoine
* | 2cbdf8b Précise le lait entier
|/
*   27189c4 Merge branch 'dessert'
|\
| * 8470f22 Ajoute la mousse au chocolat
* | 67c3f52 Précise le nombre de personnes
|/
* 8e27dd7 Ajoute la recette des gaufres
* b5bb587 Ajoute la recette des crêpes
* bed3ced Ajoute le README
```

Ici, on a gardé les deux idées. Si vous paniquez en plein conflit, `git merge --abort` annule la fusion et remet tout comme avant.

4. GitHub : mettre son dépôt en ligne

GitHub héberge vos dépôts Git sur Internet. Il sert à trois choses : sauvegarder votre travail ailleurs que sur votre machine, le montrer (votre GitHub fait partie de votre CV de développeur) et travailler à plusieurs, ce qu'on fera demain.

Créer son compte

Sur github.com, créez un compte. Choisissez un pseudo que vous pourrez montrer à un recruteur, et utilisez la même adresse mail que dans `git config`. GitHub peut vous demander d'activer la double authentification : gardez votre téléphone à côté de vous.

Se connecter avec une clé SSH

Pour que votre VM puisse envoyer du code sur GitHub, il faut qu'elle prouve qui vous êtes. GitHub refuse les mots de passe depuis le terminal. On utilise une clé SSH, comme au jour 5 de la formation Linux.

```
ssh-keygen -t ed25519 -C "alice@example.com"
cat ~/.ssh/id_ed25519.pub
```

Laissez les réponses par défaut (`Entrée` à chaque question). La deuxième commande affiche votre clé **publique**, la ligne qui commence par `ssh-ed25519`. Copiez-la, puis sur GitHub : votre photo en haut à droite, **Settings**, **SSH and GPG keys**, **New SSH key**. Collez, donnez un nom (« VM Ubuntu »), validez.

Testez la connexion :

```
ssh -T git@github.com
```

La première fois, SSH vous demande si vous faites confiance à `github.com` : tapez `yes`. Vous devez lire `Hi votre-pseudo! You've successfully authenticated, but GitHub does not provide shell access.` Ce « does not provide shell access » est normal : GitHub ne sert pas à ouvrir un terminal, juste à échanger du code.

Ne partagez jamais le fichier `id_ed25519` (sans `.pub`) : c'est votre clé privée.

Publier un dépôt existant

Sur GitHub, bouton **New** (ou le **+** en haut à droite, puis **New repository**). Donnez un nom, laissez le dépôt **vide** : ne cochez pas « Add a README », sinon votre premier envoi sera refusé. GitHub affiche alors les commandes à taper. Choisissez l'onglet **SSH**, puis dans votre dépôt local :

```
git remote add origin git@github.com:votre-pseudo/recettes.git
git push -u origin main
```

`origin` est le surnom de l'adresse GitHub de votre dépôt. `git push` envoie vos commits. Le `-u` ne sert qu'à la première fois : il relie votre branche `main` à celle de GitHub. Ensuite, `git push` tout court suffit. Rechargez la page GitHub : vos fichiers et votre historique sont là.

Récupérer un dépôt

```
git clone git@github.com:votre-pseudo/recettes.git
```

`git clone` télécharge un dépôt complet, historique compris, dans un nouveau dossier. Si le dépôt a changé sur GitHub depuis, `git pull` récupère les nouveaux commits.

Commande	Sens	Rôle
`git clone <adresse>`	GitHub → vous	télécharger un dépôt la première fois
`git push`	vous → GitHub	envoyer vos commits
`git pull`	GitHub → vous	récupérer les commits des autres

5. Un README en Markdown

Le fichier `README.md` est la vitrine du dépôt : GitHub l'affiche en dessous des fichiers. Il est écrit en **Markdown**, un format texte très simple :

````
# Titre principal
## Sous-titre

Un paragraphe avec du **gras** et de l'*italique*.

- un élément de liste
- un autre

Du code dans le texte : `bash moyenne.sh`

Un bloc de code (dans le vrai fichier, sans les quatre espaces au début des lignes) :
    ```bash
    bash moyenne.sh 12 8 15
    ```

Un lien : [le site de Git](https://git-scm.com)
````

Un bon README répond à trois questions : qu'est-ce que c'est, comment on l'utilise, qui l'a fait.

Exercices du mardi

M1. Une branche pour une nouvelle fonctionnalité NIVEAU 1

Dans votre dépôt `algo-bash` d'hier :

1. Créez une branche `ajout-rebours` et placez-vous dessus.
2. Ajoutez un script de compte à rebours (celui de vendredi dernier, ou réécrivez-le), puis commitez.
3. Revenez sur `main` et faites `ls` : le script a disparu.
4. Fusionnez la branche dans `main`, vérifiez avec `ls`, puis supprimez la branche.

M2. Deux branches qui divergent NIVEAU 2

1. Créez une branche `commentaires` et ajoutez un commentaire en haut d'un script. Commitez.
2. Revenez sur `main` et modifiez **un autre** script. Commitez.
3. Fusionnez `commentaires` dans `main`. Pourquoi Git vous demande-t-il un message, alors qu'en M1 il ne l'a pas fait ?
4. Regardez le résultat avec `git log --oneline --graph`.

M3. Le conflit volontaire NIVEAU 2

1. Sur `main`, créez un fichier `auteur.txt` avec une seule ligne : `Auteur : à compléter`. Commitez.
2. Créez une branche `prenom`, remplacez la ligne par `Auteur : votre prénom`. Commitez.
3. Revenez sur `main`, remplacez la même ligne par `Auteur : votre nom`. Commitez.
4. Fusionnez `prenom` dans `main`. Lisez le conflit avec `cat auteur.txt`.
5. Résolvez-le pour obtenir `Auteur : Prénom Nom`, puis terminez la fusion.

M4. Compte GitHub et clé SSH NIVEAU 1

Suivez la section 4. L'exercice est réussi quand `ssh -T git@github.com` répond `Hi votre-pseudo!`.

M5. Publier `algo-bash` NIVEAU 1

Créez un dépôt vide `algo-bash` sur GitHub et poussez votre dépôt local. Vérifiez sur la page GitHub que vous voyez bien vos scripts **et** l'onglet des commits (l'horloge avec le nombre de commits).

M6. Un vrai README NIVEAU 2

Écrivez le `README.md` de `algo-bash` avec :

- un titre et une phrase qui présente le dépôt ;
- une section « Lancer un script » avec un exemple dans un bloc de code ;
- la liste des scripts, avec une ligne d'explication pour chacun ;
- votre nom.

Commitez, poussez, et vérifiez le rendu sur GitHub. Vous pouvez aussi modifier le README directement sur GitHub (icône crayon). Dans ce cas, pensez à faire `git pull` sur votre VM ensuite, sinon votre prochain push sera refusé.

M7. Chez le voisin NIVEAU 3

1. Demandez l'adresse du dépôt `algo-bash` de votre voisin et clonez-le dans un dossier à part (pas dans votre propre dépôt !).
2. Lisez son historique : combien de commits ? Qui en est l'auteur ?
3. Modifiez un de ses fichiers, commitez, et essayez `git push`. Que se passe-t-il, et pourquoi ? On règle ce problème demain.

Mémo Git

Besoin	Commande
Créer un dépôt dans le dossier courant	`git init`
Où j'en suis ?	`git status` (version courte : `git status -s`)
Ajouter des fichiers au prochain commit	`git add fichier`, ou `git add .` pour tout
Enregistrer un commit	`git commit -m "Message au présent"`
Ajouter et commiter les fichiers déjà suivis	`git commit -am "Message"`
Voir ce qui a changé (pas encore ajouté)	`git diff`
Voir ce qui va partir dans le commit	`git diff --staged`
Historique	`git log --oneline`, `git log --oneline --graph --all`
Détail d'un commit	`git show <id>`
Annuler une modification non ajoutée	`git restore fichier` (perd la modification)
Sortir un fichier de l'index	`git restore --staged fichier`
Annuler un commit sans réécrire l'historique	`git revert <id>`
Lister les branches	`git branch`
Créer une branche et y aller	`git switch -c nom`
Changer de branche	`git switch nom`
Fusionner une branche dans la branche courante	`git merge nom`
Supprimer une branche fusionnée	`git branch -d nom`
Abandonner une fusion en conflit	`git merge --abort`
Relier le dépôt à GitHub	`git remote add origin git@github.com:pseudo/depot.git`
Premier envoi d'une branche	`git push -u origin nom`
Envoyer / récupérer	`git push` / `git pull`
Télécharger un dépôt	`git clone <adresse>`
Récupérer sans fusionner	`git fetch`
Qui a écrit cette ligne ?	`git blame fichier`
Quel commit a ajouté ce texte ?	`git log -S "texte" --oneline`

Dans un conflit, le fichier contient `<<<<<<< HEAD` (votre version), `=======`, puis `>>>>>>> branche` (l'autre version). Gardez ce que vous voulez, supprimez les trois marqueurs, puis `git add` et `git commit`.